// DEEP AGENTS HACKATHON 2026

Vibe Testing

Autonomous AI QA Agent Architecture — From GitHub Repo to Bug Report in 90 Seconds

SYSTEM ARCHITECTURE

Input — Any GitHub Repository

github.com/org/repo-name · No OpenAPI spec required · No test setup needed

ENTRY POINT

with swagger spec

without swagger spec ✦ NEW

any framework

✦ NEW MODULE

Spec Inference Engine — When no OpenAPI/Swagger spec exists, Macroscope reads the codebase, maps routes, controllers, and schemas, then auto-generates a valid OpenAPI 3.0 spec. This unlocks the full pipeline for **any repo**, not just documented APIs.

INGESTION & DISCOVERY PIPELINE

STEP 00 NEW

Spec Inference

Macroscope parses codebase → AST route extraction → LLM generates OpenAPI 3.0 spec automatically

Macroscope | AST Parser

STEP 01 EXISTING

Clone & Extract

Git clone repo into isolated Docker container. Extract spec (real or inferred). Parse all endpoints, schemas, metadata.

Docker | Git

STEP 02 EXISTING

Ingest & Parse

OpenAPI 3.x / Swagger 2.x / Postman v2.1 → normalized endpoint graph with types and dependencies.

ingest.py

STEP 03 EXISTING

MCP Tool Discovery

Mine capabilities from endpoints. Each API endpoint becomes an MCP tool the agent can call autonomously.

mine.py

STEP 04 EXISTING

Safety Classification

Classify endpoints by risk: READ / WRITE / DESTRUCTIVE. Apply execution policy — destructive ops require sandboxing.

safety.py

STEP 05 EXISTING

MCP Server Codegen

LLM generates a complete FastMCP server wrapping all discovered tools. Syntax validated, deps resolved.

codegen.py | FastMCP

STEP 06 SWAPPED

Deploy to TrueFoundry

Upstream API + generated MCP server deployed as live services. TrueFoundry handles scaling, health checks, and observability.

TrueFoundry | MCP Gateway

STEP 07 NEW

Aerospike Memory Init

Initialize persistent store for this repo. Load prior test runs if exists. Track bug history, fix status, regression risk across sessions. Real-time read/write during agent execution.

Aerospike | Cross-run Memory | Regression Tracking

🧠 Orchestrator Agent

Plans, reasons, and dispatches — the "thinking" layer of the system

> Analyzing repo structure... this is an **e-commerce app** with auth, cart, checkout, and inventory

> Risk ranking: **checkout flow** (payment) → **auth** (session) → **inventory** (concurrency)

> Dispatching 3 parallel sub-agents with specialized test strategies...

> Will re-evaluate after first round of results and spawn deeper probes if anomalies detected

⚡ PARALLEL SUB-AGENTS — LAUNCHED SIMULTANEOUSLY

🟢

Happy Path Agent

Tests primary user journeys end-to-end as a real user would. Validates the app works as designed.

🔶

Edge Case Hunter

Actively tries to break the app. Tests boundaries, unexpected inputs, and concurrent operations.

- Sign up → verify email → login
- Browse → add to cart → checkout
- Payment → confirmation → receipt
- Profile update → logout → re-login

3-4 layers deep

- Empty / null / special chars in inputs
- Expired card → error handling
- Max string length, negative values
- Concurrent booking race conditions

5-7 layers deep

🔴

Security Probe Agent

Tests authentication and authorization. Probes for common vulnerabilities and broken access control.

- Access admin routes without token
- Use expired / invalid JWT token
- Access another user's resources
- Brute force login rate limiting

6-8 layers deep | Auth0

▼

RE-REASONING LOOP — WHAT MAKES IT "DEEP"

👁️

Observe

Capture what happened: errors, screenshots, response codes,

→

🤔

Re-Reason

Agent reflects: "Interesting — auth failed but only on mobile

→

timing, logs

viewport. Why?"

🔍

Root Cause

Traces bug → code → exact file + line. Links to MCP tool call that exposed it.

→

💡

Suggest Fix

Generates exact code fix with explanation. Severity scored. Spawns deeper sub-agent if needed.

↺ LOOPS UNTIL AGENT IS SATISFIED — NO NEW BUGS FOUND IN LAST PASS ↺

▼

OUTPUT — FULL QA REPORT + TRUEFOUNDRY OBSERVABILITY DASHBOARD

3 CRITICAL BUGS	**5** EDGE CASE FAILURES
12 FLOWS TESTED	**90s** TOTAL RUNTIME

```
// SAMPLE OUTPUT
CRITICAL  checkout.js:47 — Unhandled promise rejection on expired card. No error shown to user.
CRITICAL  auth/route.js:23 — Admin endpoint accessible without JWT. Auth middleware missing.
WARNING   cart.js:89 — Race condition when 2 users checkout same last item simultaneously.
FIX       checkout.js:47 — Add try-catch around payment promise. Return 402 with user message.
```

SPONSOR INTEGRATIONS

🚀

TrueFoundry

Deploy MCP servers + upstream APIs. MCP Gateway for secure tool access. Live tracing dashboard showing agent reasoning in real time.

🔍

Macroscope

Codebase understanding engine. Reads repos without docs and infers the full API surface. Powers the spec inference module.

⚡

Aerospike

Real-time persistent memory across test runs. Track bug history, fix status, and regression risk over time. Sub-millisecond reads during live agent execution.

🔐

Auth0

Security probe agent tests Auth0-secured flows specifically. Validates token expiry, role-based access, and session management end-to-end.

☁️

AWS + Kiro

Cloud infrastructure for agent runtime. Kiro used to build the system itself — meta story: Kiro-built app, vibe-tested by our agent.

4-PERSON BUILD PLAN (4-5 HOURS)

PERSON 1	PERSON 2	PERSON 3	PERSON 4
Spec	TrueFoundry	Deep Agent	Demo +

Inference

Build Step 00 — Macroscope integration + AST route parser + OpenAPI generator. Test on 2 repos without swagger.

1.5 hrs

Swap

Replace Blaxel calls in backend.py + scanner.py with TrueFoundry SDK. Add tracing to agent_tester.py.

1.5 hrs

Loop

Add orchestrator agent + re-reasoning loop + root cause + fix suggestion layer on top of existing agent_tester.py.

2 hrs

Pitch

Find 2 repos with known bugs. Pre-bake demo flow. Build pitch slides. Talk to TrueFoundry + Macroscope judges.

ongoing

THE PITCH

"You've vibe coded. Now vibe test. Drop any GitHub repo — our AI agent reads your code, understands what it does, and stress tests every real user flow. What breaks, why it breaks, and how to fix it. In 90 seconds."